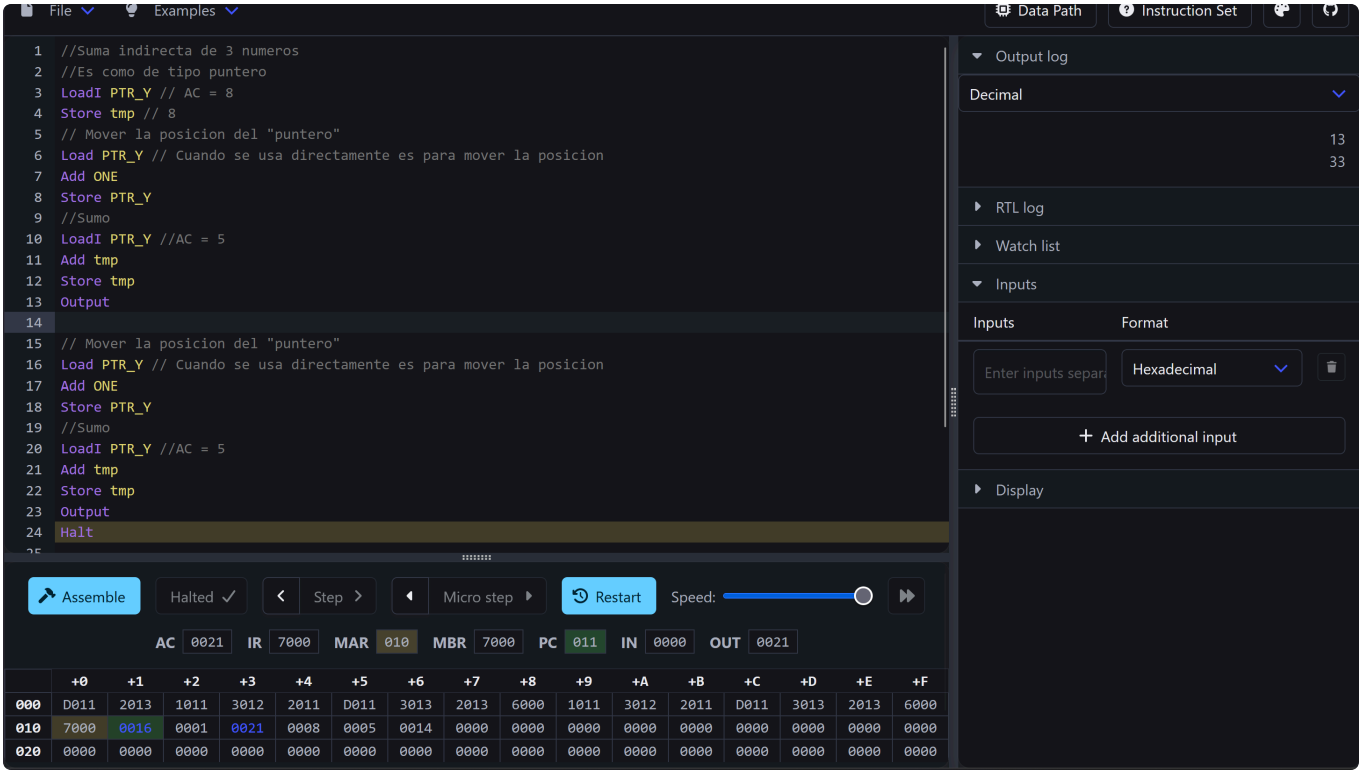
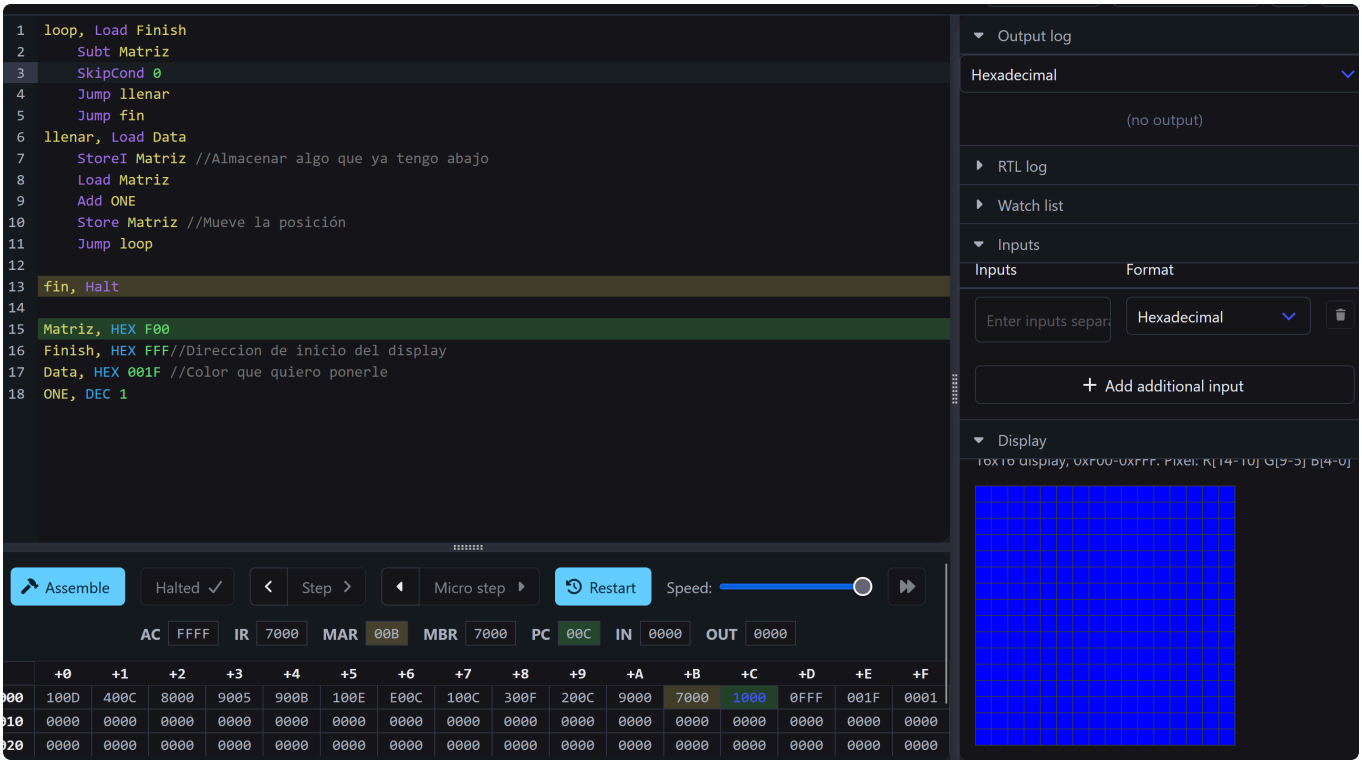
Marie tiene direct addressing mode, que significa que tienes acceso al operando directamente osea que si pones Add x, modificas directamente a x.

En la INDIRECT ADDRESSING MODE:
Se necesita saber en donde esta la dirección de Y o la variable que tenga varios valores, para a partir de allí mover los valores.

ADR: Added Data Register, nos va a dar la dirección o posición de memoria que pidamos de un dato especifico
Store del sistema a la memoria



Para llenar un display:



```
loop, Load Finish
    Subt Matriz
    SkipCond 0
    Jump llenar
    Jump fin
llenar, Load Data
    StoreI Matriz //Almacenar algo que ya tengo abajo
    Load Matriz
    Add ONE
    Store Matriz //Mueve la posición
    Jump loop

fin, Halt

Matriz, HEX F00
Finish, HEX FFF//Direccion de inicio del display
Data, HEX 001F //Color que quiero ponerle
ONE, DEC 1
```

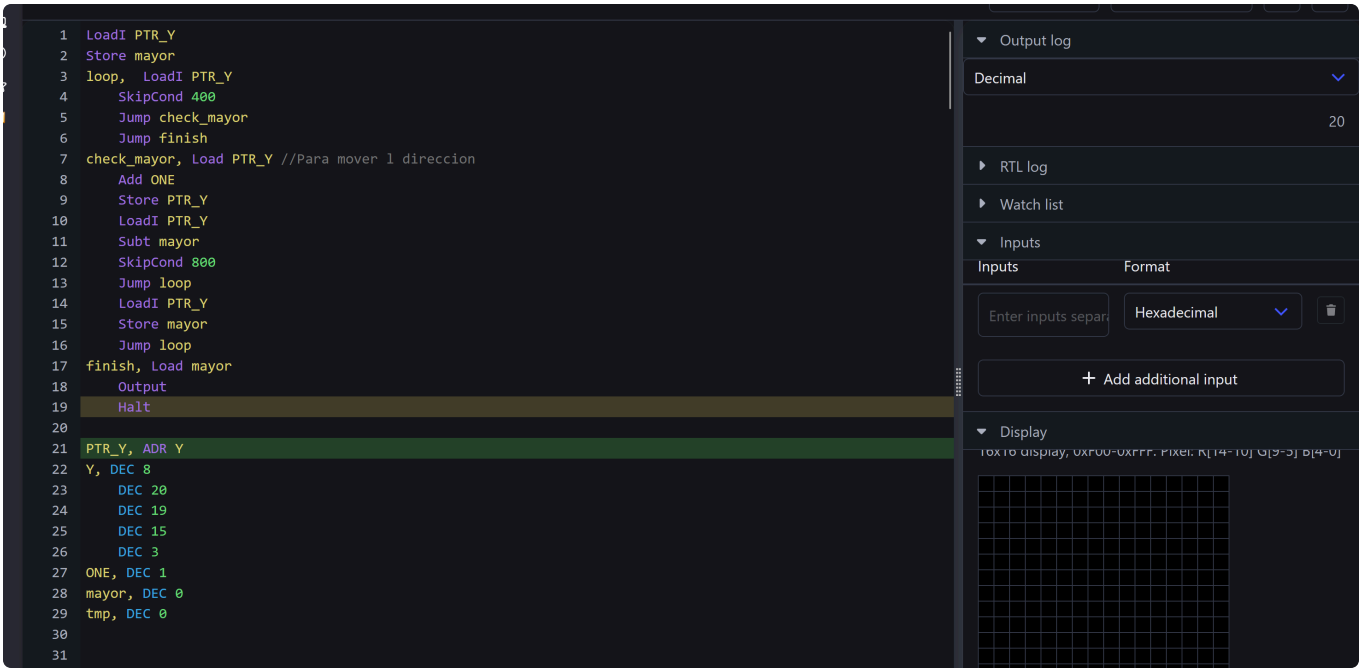
Output log
Hexadecimal
(no output)
RTL log
Watch list
Inputs
Inputs Format
Enter inputs separa
Hexadecimal
+ Add additional input
Display
16x16 display, 0xF00-0xFFF. Pixel: R[14-10] G[9-5] B[4-0]

Assemble | Halted ✓ | Step | Micro step | Restart | Speed:

AC FFFF IR 7000 MAR 00B MBR 7000 PC 00C IN 0000 OUT 0000

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
000	100D	400C	8000	9005	900B	100E	E00C	100C	300F	200C	9000	7000	1000	0FFF	001F	0001
010	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
020	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Comparacion de mayor numero:



```
LoadI PTR_Y
Store mayor
loop,  LoadI PTR_Y
    SkipCond 400
    Jump check_mayor
    Jump finish
check_mayor, Load PTR_Y //Para mover 1 direccion
    Add ONE
    Store PTR_Y
    LoadI PTR_Y
    Subt mayor
    SkipCond 800
    Jump loop
    LoadI PTR_Y
    Store mayor
    Jump loop
finish, Load mayor
    Output
    Halt

PTR_Y, ADR Y
Y, DEC 8
    DEC 20
    DEC 19
    DEC 15
    DEC 3
ONE, DEC 1
mayor, DEC 0
tmp, DEC 0
```

Output log
Decimal
20
RTL log
Watch list
Inputs
Inputs Format
Enter inputs separa
Hexadecimal
+ Add additional input
Display
16x16 display, 0xF00-0xFFF. Pixel: R[14-10] G[9-5] B[4-0]

```
1
2  ORG F00
3
4  MARIO, HEX 0000
5      HEX 0000
6      HEX 0000
7      HEX 0000
8      HEX 7C00
9      HEX 7C00
10     HEX 7C00
11     HEX 7C00
12     HEX 7C00
13     HEX 0000
14     HEX 0000
15     HEX 0000
16     HEX 0000
17     HEX 0000
18     HEX 0000
19     HEX 0000
20     HEX 0000
21     HEX 0000
22     HEX 0000
23     HEX 7C00
24     HEX 7C00
25     HEX 7C00
```

Assemble | Run | Step | Micro step | Restart | Speed:

AC 0000 IR 0000 MAR 000 MBR 0000 PC F00 IN 0000 OUT 0000

	+0	+1	+2	+3	+4	+5	+6	+7	+8	+9	+A	+B	+C	+D	+E	+F
70	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
80	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000
90	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000	0000

Output log

Decimal

(no output)

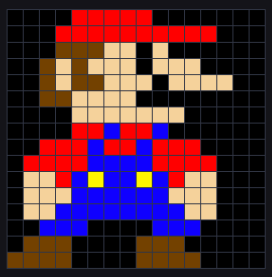
RTL log

Watch list

Inputs

Display

16x16 display, 0xF00-0xFFF. Pixel: R[14-10] G[9-5] B[4-0]



```
1  // (3+4) + (5+6)
2  Load w
3  Store sum1
4  Load x
5  Store sum2
6  JnS suma
7  Load result
8  Store tmp
9  Load y
10 Store sum1
11 Load z
12 Store sum2
13 Jns suma
14 Add tmp
15 Output
16 Halt
17
18 suma, Output
19     Load sum1
20     Add sum2
21     Store result
22     JumpI suma
23
24
25 tmp, DEC 0
26 sum1, DEC 0
27 sum2, DEC 0
28 result, DEC 0
29 w, DEC 3
30 x, DEC 4
31 y, DEC 5
```

Assemble | Halted ✓ | Step | Micro step | Restart | Speed:

AC 0012 IR 7000 MAR 00E MBR 7000 PC 00F IN 0000 OUT 0012

Output log

Decimal

18

RTL log

MAR ← PC
MBR ← M[MAR]
IR ← MBR
PC ← PC + 1
Decoded opcode 0x6 in IR[15-12] as Output
OUT ← AC

MAR ← PC
MBR ← M[MAR]
IR ← MBR
PC ← PC + 1
Decoded opcode 0x7 in IR[15-12] as Halt
Halted

Watch list

Inputs

Display

16x16 display, 0xF00-0xFFF. Pixel: R[14-10] G[9-5] B[4-0]

